Neural Networks: Coding Questions & Answer Key

```
import numpy as np
```

Question 1: Mean Squared Error (MSE)

Given `y_hat` (predictions) and `y` (true labels), write a function to calculate MSE.

Answer 1

```
def mse(y_hat, y):
    """Calculate mean squared error"""
    return np.mean((y_hat - y) ** 2)

# Test
y_true = np.array([1, 2, 3])
y_pred = np.array([1.1, 2.2, 2.9])
print(f"MSE: {mse(y_pred, y_true):.4f}")
```

```
MSE: 0.0200
```

Question 2: Sigmoid Activation Function

Implement the sigmoid activation function: σ(x) = 1 / (1 + e^(-x))

Answer 2

```
def sigmoid(x):
    """Sigmoid activation function"""
    return 1 / (1 + np.exp(-x))

# Test
x = np.array([0, 1, -1])
print(f"Sigmoid: {sigmoid(x)}")
```

```
Sigmoid: [0.5        0.73105858 0.26894142]
```

Question 3: ReLU Activation Function

Implement ReLU (Rectified Linear Unit): f(x) = max(0, x)

Answer 3

```python
def relu(x):
    """ReLU activation function"""
    return np.maximum(0, x)

# Test
x = np.array([-2, -1, 0, 1, 2])
print(f"ReLU: {relu(x)}")
```

```
ReLU: [0 0 0 1 2]
```

Question 4: Forward Pass Prediction

Given input X, weights W, and bias b, implement prediction: y_hat = X @ W + b

Answer 4

```python
def predict(X, W, b):
    """Linear forward pass prediction"""
    return X @ W + b

# Test
X = np.array([[1, 2], [3, 4]])
W = np.array([0.5, -0.5])
b = 0.1
print(f"Predictions: {predict(X, W, b)}")
```

```
Predictions: [-0.4 -0.4]
```

Question 5: Perceptron Output (Hard Threshold)

Given activation output, apply perceptron rule: y = 1 if z > 0 else -1

Answer 5

```python
def perceptron_output(z):
    """Perceptron decision boundary"""
    return np.where(z > 0, 1, -1) # or with

# Test
z = np.array([-1.5, 0, 1.5])
print(f"Outputs: {perceptron_output(z)}")
```

```
Outputs: [-1 -1  1]
```

Question 6: Sigmoid Derivative

Implement sigmoid derivative: σ'(x) = σ(x) * (1 - σ(x))

Answer 6

```
def sigmoid_derivative(x):
    """Sigmoid derivative"""
    s = sigmoid(x)
    return s * (1 - s)

# Test
x = np.array([0, 1, -1])
print(f"Derivative: {sigmoid_derivative(x)}")
```

```
Derivative: [0.25       0.19661193 0.19661193]
```

Question 7: Weight Update (Perceptron Rule)

Given misclassification error, update weights: W = W + learning_rate * error * X

Answer 7

```
def update_weights(W, X, error, learning_rate):
    """Update weights using perceptron rule"""
    return W + learning_rate * error * X

# Test
W = np.array([0.5, -0.3])
X = np.array([1, 0.5])
error = -1
lr = 0.1
new_W = update_weights(W, X, error, lr)
print(f"Updated weights: {new_W}")
```

```
Updated weights: [ 0.4  -0.35]
```

Question 8: Accuracy Calculation

Given predictions and true labels, calculate classification accuracy.

Answer 8

```
def accuracy(y_pred, y_true):
    """Calculate classification accuracy"""
    return np.mean(y_pred == y_true)
```

```
# Test
y_true = np.array([1, -1, 1, -1, 1])
y_pred = np.array([1, -1, 1, 1, 1])
print(f"Accuracy: {accuracy(y_pred, y_true):.2%}")
```

```
Accuracy: 80.00%
```

Question 9: Full Training Loop

Write a train_perceptron function that implements a complete training loop for a perceptron classifier. The function should:

Initialize weights to small random values For each epoch:

- Make predictions on all training data using the forward pass
- Convert predictions to perceptron outputs (hard threshold)
- Calculate misclassifications
- Update weights for each misclassified sample

Return final weights and a list tracking misclassifications per epoch

Parameters:

- X (numpy.ndarray): Training features of shape (n_samples, n_features)
- y (numpy.ndarray): True labels of shape (n_samples,) with values {-1, 1}
- learning_rate (float): Learning rate for weight updates
- epochs (int): Number of training iterations
- random_seed (int): For reproducibility

Returns:

- W (numpy.ndarray): Learned weights
- b (float): Learned bias

```
def train_perceptron(X, y, learning_rate=0.1, epochs=50,
random_seed=42):
    """
    Train a perceptron classifier from scratch.

    Uses: predict(), perceptron_output(), update_weights(), accuracy()
    """
    np.random.seed(random_seed)
    n_features = X.shape[1]

    # Initialize weights and bias
    W = np.random.randn(n_features) * 0.01
    b = 0.0
```

```
    for epoch in range(epochs):
        # Forward pass
        z = predict(X, W, b)  # Linear computation
        y_pred = perceptron_output(z)  # Hard threshold

        # Update weights for each misclassified sample
        for i in range(X.shape[0]):
            if y_pred[i] != y[i]:
                error = y[i] - y_pred[i]
                W = update_weights(W, X[i], error, learning_rate)
                b += learning_rate * error

    return W, b


# Test with AND gate example
X_train = np.array([[1, 1], [1, -1], [-1, 1], [-1, -1]])
y_train = np.array([1, -1, -1, -1])

W, b = train_perceptron(X_train, y_train, learning_rate=0.5,
epochs=10)

print(f"\nFinal Weights: {W}")
print(f"Final Bias: {b}")
```

```
Final Weights: [1.00496714 0.99861736]
Final Bias: -1.0
```

```
accuracy(perceptron_output(predict(X_train, W, b)), y_train)
```

```
np.float64(1.0)
```